Clean Architecture

1. ภาพรวมว่าคืออะไร

Clean Architecture คือรูปแบบการออกแบบซอฟต์แวร์ที่เน้นการแยกความรับผิดชอบออกจากกัน โดยวาง **กฎทางธุรกิจ (Business Rules)** ไว้ที่ศูนย์กลาง และทำให้ส่วนอื่นๆ เช่น ฐานข้อมูล, UI, หรือ Framework ขึ้นอยู่กับศูนย์กลางนั้นเพียงทิศทางเดียว แนวคิดนี้ได้รับการเผยแพร่โดย Robert C. Martin (Uncle Bob) โดยมีหลักการสำคัญคือ **Dependency Rule**: ซอร์สโค้ดต้องพึ่งพาเข้าด้านในเท่านั้น ไม่ใช่พึ่งพาออกด้านนอก

---

2. คำนำ

ในการพัฒนาซอฟต์แวร์สมัยใหม่ ปัญหาที่พบบ่อยคือโค้ดส่วนธุรกิจ (Business Logic) ไปผูกติดกับฐานข้อมูล, Framework หรือ UI ทำให้เมื่อต้องการเปลี่ยนเทคโนโลยี ต้องแก้ไขโค้ดจำนวนมากและเสี่ยงต่อการเกิดข้อผิดพลาด Clean Architecture ถูกออกแบบมาเพื่อแก้ปัญหานี้ โดยการแยกส่วนที่สำคัญที่สุดของระบบ (กฎธุรกิจ) ออกจากรายละเอียดที่เปลี่ยนแปลงบ่อย (เทคโนโลยี)

---

3. วัตถุประสงค์

- แยก **กฎธุรกิจ** ออกจาก **รายละเอียดทางเทคนิค** (Framework, Database, UI)
- ทำให้ระบบ **ทดสอบได้ง่าย** โดยไม่ต้องพึ่งพาฐานข้อมูลหรือ UI จริง
- ลดผลกระทบเมื่อมีการเปลี่ยนแปลงเทคโนโลยี (เช่น เปลี่ยนจาก PostgreSQL เป็น MongoDB)
- ทำให้โค้ด **บำรุงรักษาง่าย** และรองรับการเติบโตในระยะยาว

---

4. เป้าหมาย

เป้าหมายสูงสุดคือการสร้างระบบที่ **Business Logic ไม่รู้จักโลกภายนอก** ไม่ว่าจะเป็นฐานข้อมูล เว็บเฟรมเวิร์ก หรือบริการภายนอกใดๆ ทำให้แกนกลางของระบบมีความเสถียรและสามารถปรับเปลี่ยนส่วนอื่นได้อย่างอิสระ

---

5. ความรู้พื้นฐานที่จำเป็น

- **SOLID Principles** โดยเฉพาะ Dependency Inversion Principle (DIP)
- **Layered Architecture** (การแบ่งชั้นของซอฟต์แวร์)

- **Dependency Injection** (การฉีด dependency)
- **Interface / Abstraction** ในภาษาโปรแกรมที่ใช้

---

6. ข้อดี

> ข้อดี > คำอธิบาย > —>—> **Testability สูง** > ทดสอบ Business Logic ได้โดยไม่ต้องเปิดฐานข้อมูลหรือ UI > **Framework Independence** > เปลี่ยน Framework ได้โดยไม่กระทบ Business Logic > **Database Independence** > สลับฐานข้อมูลได้โดยไม่ต้องแก้ไข Domain Layer > **Maintainability** > โค้ดแยกส่วนชัดเจน บำรุงรักษาง่าย > **Decoupling** > การพึ่งพาระหว่างส่วนต่างๆ ต่ำมาก >

---

7. ข้อเสีย

- **ความซับซ้อนเริ่มต้นสูง**: ต้องสร้าง Layer และ Interface หลายชั้นตั้งแต่เริ่มต้น
- **Boilerplate เยอะ**: มีโค้ดซ้ำๆ (boilerplate) จำนวนมาก โดยเฉพาะในโปรเจกต์เล็ก
- **Overengineering สำหรับโปรเจกต์เล็ก**: อาจไม่คุ้มค่ากับเวลาและต้นทุนสำหรับระบบขนาดเล็ก
- **Learning Curve สูง**: ทีมต้องเข้าใจหลักการและวินัยในการเขียนโค้ดตามกฎ Dependency

---

8. ข้อควรระวัง

- **อย่าฝ่าฝืน Dependency Rule**: ห้ามให้ชั้นใน import ชั้นนอกเด็ดขาด (เช่น Entity import จาก Framework)
- **เหมาะกับโปรเจกต์ขนาดกลางถึงใหญ่**: หากโปรเจกต์เล็กมาก การใช้ Clean Architecture อาจสร้างภาระเกินความจำเป็น
- **ต้องมีความวินัยในทีม**: ทุกคนต้องเข้าใจและปฏิบัติตามโครงสร้างอย่างเคร่งครัด
- **เริ่มจาก Use Case ที่ชัดเจน**: ก่อนสร้าง Layer ต่างๆ ควรเข้าใจ Business Use Case ให้ถ่องแท้ก่อน

---

9. ตัวอย่าง

ระบบจองนัดหมาย (Appointment Booking System)

- **Entities**: `Appointment`, `Doctor`, `Patient` พร้อมกฎการจัดตาราง
- **Use Cases**: `BookAppointment`, `CancelAppointment`, `FindAvailableSlots`
- **Interface Adapters**: REST Controller, Presenter, Repository Interface
- **Frameworks & Drivers**: Web Framework, Database, Notification Service, Calendar API

จุดสำคัญ: กฎการจองนัดหมาย (เช่น "ห้ามจองซ้ำในเวลาเดียวกัน") อยู่ใน Entity และ Use Case **ซึ่งไม่รู้จัก**ว่าข้อมูลถูกเก็บในฐานข้อมูลอะไร หรือแสดงผลผ่าน UI แบบไหน ทำให้สามารถทดสอบกฎเหล่านี้ได้โดยไม่ต้องเปิดระบบทั้งหมด

---

10. ตัวชี้วัดเรื่องนี้ (KPI)

> KPI > คำอธิบาย > เกณฑ์สุขภาพดี > —>—>—> **Dependency Violations** > จำนวนการพึ่งพาที่ผิดทิศ (ชั้นใน import ชั้นนอก) > **0 violations** (ยิ่งน้อยยิ่งดี) > **Test Coverage (Domain & Use Cases)** > สัดส่วนโค้ดในชั้นในที่มีการทดสอบ > **≥ 50%** (ยิ่งสูงยิ่งดี) > **Layer Distribution** > สัดส่วนไฟล์ในแต่ละ Layer > Domain 15–25%, Use Cases 20–30%, Adapters 30–40%, Frameworks 15–25% > **Architecture Health Score** > คะแนนรวมสุขภาพสถาปัตยกรรม (0–100) > **90–100 = Excellent** >

---

11. การประยุกต์ใช้

Clean Architecture เหมาะกับ:

- **ระบบ Enterprise ขนาดใหญ่** ที่มีทีมพัฒนาหลายคน (20+ คน) และมีอายุการใช้งานยาวนาน (10+ ปี)
- **ระบบที่ต้องการความยืดหยุ่นสูง** ในการเปลี่ยนเทคโนโลยี (Framework, Database, UI)
- **ระบบที่มี Business Logic ซับซ้อน** และต้องการทดสอบอย่างละเอียด
- **โปรเจกต์ที่ต้องการแยก API หลายช่องทาง** (เช่น REST, GraphQL, gRPC) โดยใช้ Use Case ร่วมกัน

ไม่แนะนำ สำหรับโปรเจกต์ขนาดเล็กมากหรือ Prototype ที่ต้องการความเร็วในการพัฒนา

สรุปเป็นประโยคเดียว

> **Clean Architecture คือการจัดระเบียบโค้ดให้ "กฎธุรกิจ" อยู่ศูนย์กลาง และทุกสิ่งภายนอก (ฐานข้อมูล, UI, Framework) ต้องพึ่งพาเข้าหาศูนย์กลางเท่านั้น เพื่อให้ระบบทดสอบง่าย เปลี่ยนเทคโนโลยีได้โดยไม่กระทบหัวใจของธุรกิจ**

> **SOLID Principles:** หลักการ > ชื่อเต็ม > ความหมาย > **S** > Single Responsibility > 1 Class = 1 หน้าที่ **O** > Open/Closed > เปิดให้ขยาย ปิดให้แก้ไข **L** > Liskov Substitution > Subclass แทน Superclass ได้ **I** > Interface Segregation > Interface เล็กดีกว่าใหญ่ **D** > Dependency Inversion > ขึ้นกับ Abstraction ไม่ใช่ Concrete

เรื่อง: หลักการ SOLID (SOLID Principles)

ภาพรวมว่าคืออะไร

SOLID คือหลักการออกแบบซอฟต์แวร์เชิงวัตถุ 5 ข้อ ได้แก่ Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation และ Dependency Inversion ช่วยให้โค้ดแยกหน้าที่ชัดเจน ขยายง่าย และทดสอบง่าย

คำนำ

เมื่อระบบใหญ่ขึ้น โค้ดที่ผูกกันแน่นจะแก้ไขยากและเสี่ยงพังบ่อย SOLID ช่วยลดการพึ่งพากันเกินจำเป็นและทำให้ออกแบบคลาส/อินเทอร์เฟซได้ยืดหยุ่นขึ้น

วัตถุประสงค์

- เพื่อให้เข้าใจหลักการ SOLID ทั้ง 5 ข้อ
- เพื่อนำไปใช้แยกหน้าที่และลดการผูกติดกันของโค้ด
- เพื่อให้ออกแบบโค้ดที่ขยายและทดสอบได้ง่าย

เป้าหมาย

ออกแบบโค้ดที่อ่านง่าย แยกความรับผิดชอบชัดเจน เพิ่มฟีเจอร์ได้โดยกระทบโค้ดเดิมน้อย และพึ่งพา abstraction มากกว่า concrete

ความรู้พื้นฐานที่จำเป็น

- OOP: Class, Object, Inheritance, Polymorphism
- Interface / Abstraction
- การพึ่งพา (Dependency) และการทดสอบหน่วย

ข้อดี

- โค้ดยืดหยุ่น ขยายง่าย
- ลดผลกระทบเมื่อแก้ไข
- ทดสอบง่าย ใช้ซ้ำได้ดี
- อ่านและดูแลรักษาง่ายขึ้น

ข้อเสีย

- อาจมีคลาส/อินเทอร์เฟซมากขึ้น
- งานเล็กอาจดูซับซ้อนเกินจำเป็น
- ใช้เวลาเริ่มออกแบบมากขึ้น

ข้อควรระวัง

- ไม่ต้องใช้แบบสุดโต่งทุกจุด
- ระวัง abstraction เกินจำเป็น
- เลือกใช้ตามบริบทของงานและทีม

ตัวอย่าง

ระบบสั่งซื้อออนไลน์:

- `OrderService` ทำหน้าที่สร้างคำสั่งซื้อเท่านั้น → **S**
- เพิ่มวิธีชำระเงินใหม่ผ่าน `PaymentMethod` โดยไม่แก้โค้ดเดิม → **O, D**
- `CreditCardPayment` และ `PromptPayPayment` ใช้แทน `PaymentMethod` ได้ → **L**
- แยก `PaymentMethod` กับ `Refundable` ไม่รวมเป็นอินเทอร์เฟซใหญ่ → **I**

```
interface PaymentMethod { pay(amount) }

class CreditCardPayment implements PaymentMethod { ... }
class PromptPayPayment implements PaymentMethod { ... }

class OrderService {
  constructor(payment: PaymentMethod) {}
  checkout() { payment.pay(...) }
}
```

สรุป 1 ประโยค: SOLID คือหลักการแยกหน้าที่และพึ่งพา abstraction เพื่อให้โค้ดขยายง่าย แก้ไขน้อย และทดสอบง่าย

เรื่อง: Layered Architecture (การแบ่งชั้นของซอฟต์แวร์)

ภาพรวมว่าคืออะไร

Layered Architecture คือรูปแบบสถาปัตยกรรมซอฟต์แวร์ที่แบ่งระบบออกเป็นชั้น (Layer) แต่ละชั้นมีหน้าที่ชัดเจน เช่น ชั้นแสดงผล ชั้นธุรกิจ ชั้นเข้าถึงข้อมูล และชั้นฐานข้อมูล โดยชั้นบนเรียกใช้ชั้นล่างผ่านอินเทอร์เฟซที่กำหนด

คำนำ

เมื่อระบบเริ่มใหญ่ การเขียนโค้ดปนกันระหว่างหน้าจอ ตรรกะธุรกิจ และฐานข้อมูลทำให้แก้ไขยาก Layered Architecture ช่วยแยกความรับผิดชอบให้ชัดเจนและลดการผูกติดกันระหว่างส่วนต่าง ๆ

วัตถุประสงค์

- เพื่อแยกหน้าที่ของแต่ละชั้นให้ชัดเจน
- เพื่อลดการพึ่งพากันระหว่าง UI, Business Logic และ Data Access
- เพื่อให้ระบบดูแลรักษาและทดสอบได้ง่ายขึ้น

เป้าหมาย

ออกแบบระบบที่แต่ละชั้นเปลี่ยนแทนกันได้ง่ายขึ้น เช่น เปลี่ยนฐานข้อมูลโดยกระทบชั้นธุรกิจน้อยที่สุด และสามารถทดสอบแต่ละชั้นแยกกันได้

ข้อดี

- แยกหน้าที่ชัดเจน อ่านและดูแลง่าย
- เปลี่ยนเทคโนโลยีแต่ละชั้นได้ง่ายขึ้น
- ทดสอบแต่ละชั้นแยกกันได้
- เหมาะกับระบบองค์กรทั่วไปและทีมที่แบ่งงานตามชั้น

ข้อเสีย

- อาจมีโค้ดส่งต่อระหว่างชั้นมากเกินไป
- งานเล็กอาจดูซับซ้อนเกินจำเป็น
- ถ้าออกแบบไม่ดี ชั้นอาจรั่วไหล (Leaky Layer) เช่น UI ไปเรียกฐานข้อมูลตรง

ข้อควรระวัง

- ไม่ให้ชั้นบนข้ามไปเรียกชั้นล่างเกินลำดับที่กำหนด
- ระวังชั้นธุรกิจผูกกับรายละเอียดฐานข้อมูลมากเกินไป
- หลีกเลี่ยงการสร้างชั้นเยอะเกินความจำเป็น
- ใช้ DTO/Interface เพื่อลดการรั่วไหลของรายละเอียดระหว่างชั้น

ตัวอย่าง

ระบบสั่งซื้อออนไลน์แบ่งเป็น 4 ชั้น:

1. **Presentation Layer** – รับคำขอจากผู้ใช้ เช่น หน้า checkout
2. **Business Layer** – ตรวจสอบสต็อก คำนวณราคา และสร้างคำสั่งซื้อ
3. **Data Access Layer** – บันทึก/อ่านข้อมูลคำสั่งซื้อ
4. **Database Layer** – เก็บข้อมูลจริง

```
UI → OrderService → OrderRepository → Database
```

เมื่อผู้ใช้กดสั่งซื้อ:

- UI ส่งข้อมูลไปที่ `OrderService`
- `OrderService` ตรวจสอบกฎธุรกิจ
- `OrderRepository` บันทึกลงฐานข้อมูล
- UI ไม่รู้รายละเอียด SQL หรือฐานข้อมูล

สรุป 1 ประโยค: Layered Architecture คือการแบ่งระบบเป็นชั้นตามหน้าที่ เพื่อให้แก้ไข เปลี่ยนเทคโนโลยี และทดสอบได้ง่ายขึ้นโดยลดการผูกติดกันระหว่างส่วนต่าง ๆ

เรื่อง: Layer Distribution (การกระจายชั้นในสถาปัตยกรรมซอฟต์แวร์)

ภาพรวมว่าคืออะไร

Layer Distribution หมายถึง การตัดสินใจว่า “Layer” (ชั้นลอจิกของโค้ด) แต่ละชั้นจะถูกนำไปติดตั้ง (Deploy) ไว้บน “Tier” (ชั้นฟิสิกส์/เซิร์ฟเวอร์) ใดบ้าง แม้คำว่า Layer และ Tier จะถูกใช้สลับกันบ่อย แต่ในทางสถาปัตยกรรมถือเป็นคนละเรื่อง: **Layering** คือการจัดระเบียบโค้ดเชิงตรรกะ ส่วน **Tiering** คือการกระจายโค้ดนั้นไปยังขอบเขตฟิสิกส์

คำนำ

เมื่อออกแบบระบบแบบ Layered Architecture แล้ว คำถามสำคัญที่ตามมาคือ “จะเอา Layer ไหนไปวางบนเครื่องไหน” การกระจายชั้นจึงเป็นสะพานเชื่อมระหว่างการออกแบบซอฟต์แวร์กับการติดตั้งใช้งานจริง หากตัดสินใจผิด ระบบอาจสูญเสียประสิทธิภาพ ดูแลยาก หรือมีต้นทุนโครงสร้างพื้นฐานสูงเกินจำเป็น

วัตถุประสงค์

- เพื่อแยกความแตกต่างระหว่าง **Logical Layer** กับ **Physical Tier** ให้ชัดเจน
- เพื่อเป็นแนวทางเลือกกลยุทธ์การกระจายชั้น (Distribution Strategy) ที่เหมาะกับข้อจำกัดด้านประสิทธิภาพ ความปลอดภัย และ Scalability
- เพื่อให้ทีมสถาปัตยกรรมและทีมปฏิบัติการ (Ops) สื่อสารกันได้ตรงกัน

เป้าหมาย

กำหนดว่า Layer ใดควรอยู่ใน Tier เดียวกัน และ Layer ใดควรแยกออกไปเพื่อลด Coupling, เพิ่มความสามารถในการขยายตัว หรือตอบสนองข้อกำหนดด้านความปลอดภัย โดยไม่ทำให้สถาปัตยกรรมซับซ้อนเกินความจำเป็น

ข้อดี

- **ประสิทธิภาพดีขึ้นเมื่อจัดวางเหมาะสม**: ลดการเรียกข้าม Process/Network ที่ไม่จำเป็น
- **ขยายระบบได้ยืดหยุ่น**: แยก Tier ที่รับภาระหนักออกไปขยายได้อิสระ
- **ความปลอดภัยดีขึ้น**: ชั้นข้อมูลสามารถอยู่ใน Tier ที่เข้าถึงยากขึ้นได้
- **ต้นทุนควบคุมได้**: เลือกจำนวน Tier ให้พอดีกับความต้องการจริง

ข้อเสีย

- **เพิ่มความซับซ้อนในการ Deploy**: ยิ่งมี Tier มาก ยิ่งต้องจัดการ Build, Config และ Network ให้มากขึ้น
- **Latency สูงขึ้น**: การเรียกข้าม Tier มีค่าใช้จ่ายด้านเวลาเสมอ
- **ต้นทุนโครงสร้างพื้นฐานเพิ่ม**: แต่ละ Tier ต้องมีฮาร์ดแวร์/คลาวด์รองรับ
- **ทดสอบยากขึ้น**: ปัญหาอาจเกิดได้ทั้งจากโค้ดและจาก Network ระหว่าง Tier

ข้อควรระวัง

- **อย่า Map Layer ต่อ Tier แบบ 1:1 โดยอัตโนมัติ**: Layer ที่แยกกันเชิงตรรกะไม่จำเป็นต้องอยู่คนละเซิร์ฟเวอร์เสมอ
- **ระวัง "Leaky Tier"**: ข้อมูลหรือการเรียกที่รั่วข้าม Tier มากเกินไปจะทำให้สถาปัตยกรรมเปราะบาง
- **เลือก Distribution แนวตั้ง (Vertical) หรือแนวนอน (Horizontal) ให้ถูกบริบท**: Vertical คือการ Map Layer ลง Tier, Horizontal คือการแบ่งข้อมูล (Sharding)
- **คำนึงถึง Performance Overhead**: ทุกครั้งที่เพิ่ม Tier จะมี Cross-process/Croos-boundary call ตามมา

ตัวอย่าง

ระบบ E-Commerce แบบ 3 Layer (Presentation, Business, Data) เลือกกระจายดังนี้:

- **Single-Tier (Monolith)**: ทั้ง 3 Layer อยู่บนเซิร์ฟเวอร์เดียว → เรียบง่าย แต่ขยายยาก
- **Two-Tier (Client–Server)**: Presentation อยู่บน Client, Business + Data อยู่บน Server → เหมาะกับระบบที่ต้องการ Centralize ข้อมูล
- **Three-Tier**: แยก Presentation (Web Server), Business (App Server), Data (DB Server) ออกจากกัน → ยืดหยุ่นที่สุด แต่ Latency และต้นทุนสูงขึ้น

> ระบบที่โตเร็วและต้องการแยกส่วนขยาย มักเลือก Three-Tier หรือ N-Tier; ระบบเล็กที่ต้องการความเรียบง่ายอาจเลือก Single-Tier หรือ Two-Tier

สรุป 1 ประโยค: Layer Distribution คือการตัดสินใจว่า Layer เชิงตรรกะจะถูกวางบน Tier ฟิสิกส์ใด โดยต้องสร้างสมดุลระหว่างความเรียบง่าย ประสิทธิภาพ ความปลอดภัย และความสามารถในการขยายตัว ไม่ใช่แค่ Map 1 Layer ต่อ 1 Server ตามสัญชาตญาณ

เรื่อง: Architecture Health Score (คะแนนสุขภาพสถาปัตยกรรม)

ภาพรวมว่าคืออะไร

Architecture Health Score คือคะแนนหรือชุดตัวชี้วัดที่ใช้วัดว่าสถาปัตยกรรมซอฟต์แวร์ "แข็งแรง" แค่ไหนในมิติต่าง ๆ เช่น การแยกโมดูล การพึ่งพากัน ความซับซ้อน ความสามารถในการทดสอบ ความปลอดภัย และหนี้ทางเทคนิค โดยมักสรุปเป็นคะแนนเดียว (เช่น 0–100) เพื่อใช้ติดตามแนวโน้มและ Prioritize งานปรับปรุง

คำนำ

สถาปัตยกรรมที่ดูดีในวันออกแบบอาจเสื่อมสภาพเมื่อระบบโตขึ้นและมีคนแก้โค้ดหลายทีม คะแนนสุขภาพสถาปัตยกรรมช่วยให้ทีมเห็น “สัญญาณเตือน” ก่อนปัญหาจะกลายเป็นวิกฤต โดยเปลี่ยนจากการตัดสินด้วยความรู้สึกมาเป็นการวัดและติดตามอย่างเป็นระบบ

วัตถุประสงค์

- เพื่อวัดสุขภาพสถาปัตยกรรมอย่างเป็นระบบและติดตามแนวโน้มตามเวลา
- เพื่อระบุจุดเสี่ยง เช่น Coupling สูง, Modularity ต่ำ, Dependency ผิดทิศ
- เพื่อใช้จัดลำดับความสำคัญของงานปรับปรุงสถาปัตยกรรม

เป้าหมาย

ได้คะแนนและรายงานที่สื่อสารกับทีมเทคนิคและผู้มีส่วนได้ส่วนเสียได้ง่ายขึ้น โดยคะแนนต้องสะท้อนบริบทของระบบ ไม่ใช่ตัวเลขลอย ๆ และนำไปสู่การตัดสินใจปรับปรุงที่ชัดเจน

ข้อดี

- มองเห็นปัญหาสถาปัตยกรรมได้เร็วและเป็นกลางขึ้น
- ใช้ติดตามผลหลังปรับปรุงได้
- ช่วย Prioritize งานลดหนี้ทางเทคนิค
- สื่อสารสุขภาพระบบกับผู้บริหารได้ง่าย

ข้อเสีย

- คะแนนอาจทำให้เข้าใจผิดหากใช้นิยาม/น้ำหนักไม่เหมาะ
- คุณภาพขึ้นกับข้อมูลที่ใช้วัด
- อาจถูก “เล่นเกม” เพื่อให้คะแนนดีแต่ไม่ได้แก้ปัญหาจริง
- บางมิติ เช่น ความปลอดภัยหรือความ maintainability วัดได้ยาก

ข้อควรระวัง

- อย่าใช้คะแนนเดียวตัดสินทุกอย่าง ต้องดูรายมิติประกอบ
- กำหนดนิยาม น้ำหนัก และเกณฑ์ให้ชัดก่อนวัด
- คะแนนต้องอ้างอิงบริบท เช่น ระบบเล็กกับระบบองค์กรใช้เกณฑ์ต่างกัน
- ระวังตัวชี้วัดที่วัดง่ายแต่ไม่สะท้อนคุณภาพจริง
- ใช้เป็นเครื่องมือสนทนาและตัดสินใจ ไม่ใช่เป้าหมายในตัวเอง

ตัวอย่าง

ระบบ E-Commerce วัดสุขภาพสถาปัตยกรรม 5 มิติ (ถ่วงน้ำหนัก):

มิติ	น้ำหนัก	คะแนน (0–100)
Modularity / Cohesion	25%	70
Coupling / Dependency	25%	60
Testability	20%	80
Security	15%	75
Technical Debt	15%	50

คำนวณได้: `(70×0.25) + (60×0.25) + (80×0.20) + (75×0.15) + (50×0.15) = 66.25`

ตีความว่า สุขภาพโดยรวมปานกลาง จุดที่ควรปรับก่อนคือ Technical Debt และ Coupling แม้ Testability จะดีก็ตาม

สรุป 1 ประโยค: Architecture Health Score คือคะแนนวัดสุขภาพสถาปัตยกรรมหลายมิติที่ช่วยให้ทีมเห็นจุดเสี่ยงและจัดลำดับการปรับปรุง โดยต้องดูรายมิติและบริบทประกอบเสมอ ไม่ใช่ยึดตัวเลขเดียวเป็นสรุปสุดท้าย